Instructivo de Práctica — Sesión 10: Calidad, CI/CD y código con agentes de IA

> 📌 **Qué vas a entregar:** al terminar esta práctica, ExploraEC captura los errores que nadie esperaba, tiene pruebas de widgets, pasó una auditoría de secretos y tiene una **puerta de calidad (CI) en GitHub Actions**. Además, un **agente de código** (Claude Code, Codex o Antigravity), trabajando dentro de un **arnés** con reglas y permisos, le agrega a la pantalla Gastos un **filtro por categoría**; la CI lo verifica y un **release automático** publica el APK, que instalas en el emulador.

Objetivo

Aplicar a ExploraEC el recorrido completo de una app entregable: capturar los errores inesperados, probar con pruebas de widgets, auditar secretos, poner una puerta de calidad automática en GitHub Actions, configurar el arnés de un agente de código (archivo de contexto y permisos), pedirle un cambio pequeño con un buen pedido, verificarlo y publicar el APK con un release automático.

Requisitos previos

- Proyecto `exploraec` de la Sesión 9, guardado en git y **subido a tu repositorio de GitHub** (el remoto `origin` ya existe desde las sesiones anteriores), con permiso para escribir y para usar **Actions**.
- Docker y el repositorio del backend `proyecto-curso-spec-kit` (el Paso 0 trae los comandos para encenderlo).
- Un emulador Android creado.
- Un agente de código instalado y con sesión iniciada. La práctica se escribe con **Claude Code** (la cuenta de la Sesión 1; en pareja si hace falta). Si usas **Codex** o **Antigravity** (`agy`), la tabla del Paso 6 trae los comandos equivalentes. *Gemini CLI ya no sirve: Google lo reemplazó por Antigravity.*
- Node.js ya no hace falta hoy.

Cómo trabajar esta práctica

Los **Pasos 2, 3 y 10** son iguales a las sesiones anteriores: el código ya está en la rama y **tú solo descomentas** los bloques `TODO(sesion-10) Paso N` (selecciona las líneas comentadas que están **debajo** del `TODO`, presiona `Ctrl + /` —`Cmd + /` en Mac— y después `R` en la terminal de `flutter run`). Los **Pasos 5 a 9** son trabajo real con Git, GitHub y un agente: no hay nada que descomentar.

Usarás cuatro terminales:

Terminal	Carpeta	Para qué
A — backend	`exploraec-curso/proyecto-curso-spec-kit`	`docker compose up` (queda abierta)
B — app	`exploraec-curso/exploraec`	`flutter run` (queda abierta)
C — comandos	`exploraec-curso/exploraec`	`git`, `flutter test`, `grep`, `adb`
D — agente	`exploraec-curso/exploraec`	el agente de código (desde el Paso 6)

Tiempo estimado

Total: 60 minutos

Paso	Descripción	Tiempo
0	Encender todo desde cero y recap de la Sesión 9	5 min
1	Traer la rama de hoy y medir la línea base	3 min
2	Errores que nadie esperaba	5 min
3	Pruebas de widgets y forzar una falla	5 min
4	Auditoría de secretos y registros	4 min
5	CI: la puerta de calidad en GitHub Actions	5 min
6	El arnés del agente: contexto y permisos	6 min

Paso	Descripción	Tiempo
7	Código con un agente: filtro por categoría	9 min
8	Entrega continua: integrar y publicar `v1.0.0`	7 min
9	Mientras se publica; luego, descargar e instalar el APK	11 min
10 *(opcional)*	«Acerca de» y licencias — no cuenta dentro de los 60 min	6 min
11 *(opcional)*	Un *hook* que formatea tras cada edición — no cuenta dentro de los 60 min	6 min

> **Ojo con las esperas:** la CI tarda unos **2 a 3 minutos** y el release, unos **10** la primera vez (casi todo es compilar el APK). Por eso los pasos 5 y 8 **arrancan una ejecución y siguen con otra cosa**; vuelves a mirar el resultado cuando termina.

Paso 0 — Encender todo desde cero y recap de la Sesión 9 (5 min)

Por qué se hace

Para cerrar bien una app hay que verla funcionando: backend, emulador y app encendidos. Este paso se hace con tu proyecto **tal como quedó en la Sesión 9**, antes de traer la rama de hoy.

Instrucciones

0.1 — Guardar la Sesión 9. En la terminal C:

```
git status
```

Si aparecen cambios sin guardar, guárdalos (el Punto de partida sobrescribe `lib/`):

```
git add -A
git commit -m "Sesión 9"
```

0.2 — Levantar el backend. En la terminal A, dentro de `proyecto-curso-spec-kit`, y déjala abierta:

```
docker compose up
```

Termina cuando veas `Uvicorn running on http://0.0.0.0:8000`.

0.3 — Comprobar el backend. En la terminal C:

```
curl http://localhost:8000/health
```

Debe responder `{"status":"ok","db":"ok"}`.

0.4 — Encender el emulador. En la terminal C (`<id>` es la primera columna de la lista):

```
flutter emulators
flutter emulators --launch <id>
```

0.5 — Ejecutar tu app. En la terminal B, con el emulador encendido (y déjala abierta):

```
flutter run
```

0.6 — Entrar a tus gastos. En la app: pestaña **Gastos**, inicia sesión y comprueba que ves tu lista.

0.7 — Responder en una frase: hoy un agente de IA escribirá parte de tu app. ¿Qué te asegura que lo que escriba funciona y no filtra un secreto? Lo irás armando paso a paso.

Resultado esperado

Backend, emulador y app encendidos, con tu sesión iniciada y tu lista de gastos visible.

Errores comunes

- **`Cannot connect to the Docker daemon`**. *Solución:* abrir Docker Desktop y esperar a que diga *running*; en Linux, `sudo systemctl start docker`.
- **`flutter emulators` no lista ninguno**. *Solución:* crea uno en Android Studio (*Device Manager*), como en la Sesión 5.
- **La pestaña Gastos pide iniciar sesión con «Tu sesión caducó»**. *Solución:* el token dura 30 minutos; inicia sesión de nuevo.

✅ Antes de continuar, confirma que...

- ☐ `curl http://localhost:8000/health` responde `ok`.
- ☐ Tu app corre y ves tu lista de gastos.
- ☐ La Sesión 9 está guardada en un commit.

Punto de partida

Dentro de tu proyecto `exploraec`, en la terminal C:

```
git fetch starter
git checkout starter/sesion-10 -- lib test pubspec.yaml PLACEHOLDERS.md docs .github
.claude AGENTS.md CLAUDE.md analysis_options.yaml
flutter pub get
```

La rama agrega un solo paquete, `flutter_localizations`, que viene con el propio SDK de Flutter (para que los textos del sistema, como la pantalla de licencias, salgan en español). Trae el manejo de errores (con su bloque por descomentar), las pruebas, los dos workflows de GitHub Actions (`ci.yml` y `release-apk.yml`), el archivo de contexto del agente (`AGENTS.md` y `CLAUDE.md`), sus permisos (`.claude/`) la carpeta `docs/` (guía de arneses, plantilla de `README.md` y lista de verificación) y un `analysis_options.yaml` que es el de `flutter create` con una línea más: excluye `lib/playground/` (los ejercicios de la Sesión 1, que usan `print` a propósito) para que el análisis no marque avisos que no son de la app. El detalle está en `PLACEHOLDERS.md`.

Paso 1 — Traer la rama de hoy y medir la línea base (3 min)

Por qué se hace

Antes de mejorar algo hay que medirlo. Hoy la línea base son las pruebas: ¿cuántas pasan antes de tocar nada?

Instrucciones

1.1 — Traer la rama. Ejecutar el **Punto de partida** de arriba.

1.2 — Contar las pruebas. En la terminal C:

```
flutter test
```

Deben dar **24 pasando y 1 fallando**. La que falla es la del Paso 2: todavía no conectas el manejo de errores.

1.3 — Ver lo que trajo la rama. En la terminal C (en PowerShell: `dir -Force`):

```
ls -a
```

1.4 — Reiniciar la app. En la terminal B presiona `R`.

Resultado esperado

`flutter test` da 24 pasando y 1 fallando. Ves `AGENTS.md`, `CLAUDE.md` y las carpetas `.github`, `.claude` y `docs`.

Errores comunes

- **`flutter test` falla por un error de compilación**. *Solución:* confirmar que `flutter pub get` se ejecutó después del Punto de partida.
- **No aparecen `.github` ni `.claude`**. *Solución:* son carpetas ocultas (empiezan con punto): usa `ls -a` en Linux/Mac, o «mostrar archivos ocultos» en Windows.

✅ Antes de continuar, confirma que...

- ☐ `flutter test` muestra 24 pasando y 1 fallando.
- ☐ Ves `AGENTS.md`, `.github` y `.claude`.

Paso 2 — Errores que nadie esperaba (5 min)

Por qué se hace

Los errores que programaste (sin conexión, 400, 401) tienen mensaje propio. Pero siempre hay uno que nadie previó: un widget que falla al construirse, o un `Future` sin `catch`. Una app robusta los captura en **un solo lugar**, los registra **sin guardar secretos** y le muestra algo amistoso a la persona.

Instrucciones

2.1 — Ver el problema. En la app: pestaña **Gastos**, menú ⋮ → **Error de pantalla (solo práctica)**. Aparece la pantalla roja de depuración. Vuelve con el botón atrás del emulador.

2.2 — Descomentar. En `lib/utils/reporte_errores.dart`, descomentar el bloque `TODO(sesion-10) Paso 2` (10 líneas, dentro de `instalar()`).

2.3 — Aplicar. En la terminal B, presionar `R`.

2.4 — Repetir el error de pantalla. Menú ⋮ → **Error de pantalla (solo práctica)**: ahora aparece «Esta pantalla tuvo un problema», sin trazas técnicas.

2.5 — Repetir el error asíncrono. Vuelve atrás y usa **Error asíncrono (solo práctica)**: aparece el aviso «Algo salió mal» y en la terminal B la línea `[ExploraEC] error no controlado -> …`.

2.6 — Comprobar con las pruebas. En la terminal C:

```
flutter test
```

Resultado esperado

Los dos errores terminan en un solo lugar: la persona ve una vista amistosa o un aviso, y tú ves una línea limpia en la consola. `flutter test` da **25 pasando y 0 fallando**.

Errores comunes

- **Sigue apareciendo la pantalla roja**. *Solución:* falta descomentar el bloque o presionar `R` (mayúscula). Las líneas deben quedar sin `//`.
- **El aviso «Algo salió mal» no aparece**. *Solución:* mira la terminal B: si ves la línea `[ExploraEC]`, el error sí se capturó.
- **`Undefined name 'ErrorPantallaView'`**. *Solución:* no borres los imports de la parte superior del archivo.

✅ Antes de continuar, confirma que...

- ☐ La pantalla roja fue reemplazada por «Esta pantalla tuvo un problema».
- ☐ El error asíncrono dejó una línea `[ExploraEC]` en la terminal B.
- ☐ `flutter test` da 25 pasando.

Paso 3 — Pruebas de widgets y forzar una falla (5 min)

Por qué se hace

Una prueba de widget arma una pantalla, escribe y toca como lo haría una persona, y comprueba lo que se ve. Sirve para que una mejora de hoy no rompa lo de ayer, y será **lo que verifique el código que escriba el agente**. Estas tres no necesitan el backend ni el emulador: usan un servidor falso. Después fuerzas una falla real (apagar el backend) para ver el estado de error.

Instrucciones

3.1 — Descomentar. En `test/sesion_10_test.dart`, descomentar el bloque `TODO(sesion-10) Paso 3` (27 líneas, al final del archivo).

3.2 — Ejecutar. En la terminal C:

```
flutter test
```

3.3 — Apagar el backend. En la terminal A presiona `Ctrl + C`.

3.4 — Forzar la falla. En la app, pestaña **Gastos**, toca el ícono de recargar: aparece el aviso «Sin conexión — gastos guardados» con tu lista (la caché de la Sesión 7).

3.5 — Volver a encender el backend. En la terminal A, dentro de `proyecto-curso-spec-kit`:

```
docker compose up
```

Resultado esperado

Con el bloque descomentado, `flutter test` da **28 pasando y 0 fallando**. Con el backend apagado, Gastos no se queda en blanco: avisa que usa la caché. Al volver a encenderlo, la app se recupera.

Errores comunes

- **`flutter test` da errores de sintaxis**. *Solución:* quedó alguna línea del bloque con `//`: descomenta hasta la última línea (la llave `}` final de la tercera prueba).
- **Fallan más pruebas de las esperadas**. *Solución:* falta el bloque del Paso 2; las pruebas del Paso 3 usan `ReporteErrores.instalar()`.
- **`docker compose up` dice `no configuration file provided`**. *Solución:* ejecútalo dentro de

`proyecto-curso-spec-kit`.

✅ Antes de continuar, confirma que...

- ☐ `flutter test` da 28 pasando.
- ☐ Viste el aviso de caché con el backend apagado.
- ☐ El backend quedó encendido de nuevo.

Paso 4 — Auditoría de secretos y registros (4 min)

Por qué se hace

Antes de publicar nada hay que comprobar que no sale ningún secreto: lo que se sube a GitHub queda publicado. Se revisa que no haya claves en el código, que `.env` no se suba nunca y que el token no aparezca en ningún registro. Hoy, además, hay un **nuevo lugar donde cuidar secretos**: lo que le pasas a un agente de IA.

Instrucciones

4.1 — Buscar claves. En la terminal C (en PowerShell: `Get-ChildItem -Recurse lib,test,.github | Select-String -Pattern "api_key|sk-|SECRET_KEY"`):

```
grep -rniE "api[_-]?key|sk-[a-z0-9]{10}|SECRET_KEY" lib test .github AGENTS.md
```

No debe imprimir nada.

4.2 — Buscar tokens en registros.

```
grep -rnE "(debugPrint|print)\(.*(token|Bearer)" lib
```

No debe imprimir nada.

4.3 — Proteger lo que no se sube. Abre el archivo `.gitignore` de tu proyecto y agrega al final estas tres líneas (una por línea). Guarda el archivo:

```
.env
.claude/settings.local.json
CLAUDE.local.md
```

4.4 — Comprobar la regla.

```
git check-ignore -v .env
```

Debe mostrar `.gitignore:` seguido de un número de línea y `.env`.

4.5 — Comprobar el historial.

```
git log --all --oneline -- .env
```

No debe imprimir nada: `.env` nunca se subió.

Resultado esperado

Ninguna clave, ningún token en registros, `.env` ignorado y sin historial. Sabes que `limpiar()` (en `lib/utils/reporte_errores.dart`) quita los tokens de los mensajes antes de registrarlos.

Errores comunes

- **El `grep` de claves encuentra algo**. *Solución:* revisa si es una clave real o solo la palabra en un comentario. Una clave real se borra del código y se **rota** (se cambia) en el servicio que la emitió; borrarla no basta si ya se subió.
- **`git check-ignore` no imprime nada**. *Solución:* la línea `.env` no quedó guardada en `.gitignore`.
- **`git log` muestra commits**. *Solución:* un `.env` se subió alguna vez; hay que rotar sus secretos y limpiar el historial (pídelo al docente).

✅ Antes de continuar, confirma que...

- ☐ Los dos `grep` no encontraron nada.
- ☐ `git check-ignore -v .env` muestra la regla.
- ☐ El historial no contiene ningún `.env`.

Paso 5 — CI: la puerta de calidad en GitHub Actions (5 min)

Por qué se hace

Hasta ahora las pruebas dependen de que **tú** las ejecutes. La **integración continua (CI)** las ejecuta sola, en una máquina limpia, cada vez que subes código. Si algo falla, GitHub lo marca con una ✗ roja **antes** de que el cambio entre. Hoy es la puerta que verificará también lo que escriba el agente.

Instrucciones

5.1 — Leer el workflow. Abre `.github/workflows/ci.yml` y ubica sus tres partes: cuándo corre (`on`: cada push y cada pull request), en qué máquina (`runs-on`) y los pasos, entre ellos **Analizar el código** y **Ejecutar las pruebas**.

5.2 — Crear una rama y subirla. En la terminal C:

```
git checkout -b cierre-s10
git add -A
git commit -m "Sesión 10: errores, pruebas, auditoría y CI"
git push -u origin cierre-s10
```

5.3 — Hacer fallar una prueba a propósito, en otra rama. Crea la rama y rompe una prueba:

```
git checkout -b prueba-roja
```

En `test/sesion_10_test.dart`, en la prueba de la contraseña corta, cambia `'abc'` por `'abcdefgh'` y guarda. Luego:

```
git commit -am "Prueba rota a propósito"
git push -u origin prueba-roja
```

5.4 — Volver a tu rama. El cambio roto se queda en su rama:

```
git checkout cierre-s10
```

5.5 — Mirar los resultados. Sigue con el Paso 6 mientras las ejecuciones terminan (unos 2 a 3 minutos). Después, en tu repositorio en GitHub, abre la pestaña **Actions**: la ejecución de `cierre-s10` debe estar en verde (✓) y la de `prueba-roja` en rojo (✗).

5.6 — Leer un fallo. Abre la ejecución roja y entra al paso **Ejecutar las pruebas**: dice qué prueba falló, qué esperaba y qué encontró. Es el mismo mensaje que verías con `flutter test` en tu computador.

5.7 — Limpiar. Borra la rama de prueba:

```
git branch -D prueba-roja
git push origin --delete prueba-roja
```

Resultado esperado

Una ejecución «CI» en verde para `cierre-s10` y otra en rojo para `prueba-roja`, cuyo detalle muestra la prueba que falló. Tu rama de trabajo no tiene el cambio roto.

Errores comunes

- **`git push` pide credenciales o falla con `rejected`**. *Solución:* usa tu token de GitHub, no tu contraseña; si dice `rejected`, haz `git pull --rebase` y vuelve a subir.
- **No aparece ninguna ejecución en Actions**. *Solución:* en GitHub, *Settings* → *Actions* → *General* debe permitir ejecutar acciones; el archivo `ci.yml` debe estar dentro del commit que subiste (`git show --stat HEAD`). Si todo está bien y no aparece nada, lánzala a mano: pestaña **Actions → CI → Run workflow**.
- **La ejecución falla en «Analizar el código»**. *Solución:* corre `flutter analyze` en tu computador y corrige lo que muestre (la CI solo falla con errores y advertencias, no con sugerencias de estilo).
- **Se canceló una ejecución**. *Solución:* es normal: si subes dos veces seguidas a la **misma** rama, la primera se cancela.

✅ Antes de continuar, confirma que...

- ☐ La rama `cierre-s10` está en GitHub y su ejecución «CI» terminó en verde (o está en curso).
- ☐ Viste una ejecución en rojo y leíste qué prueba falló.
- ☐ Estás en la rama `cierre-s10` y la rama de prueba se borró.

Paso 6 — El arnés del agente: contexto y permisos (6 min)

Por qué se hace

Un agente de código es un modelo de IA más su **arnés**: el programa que lo rodea y que le da herramientas, permisos y reglas. Lo que decide cómo se comporta en **tu** proyecto es el **archivo de contexto** (las reglas que lee al empezar) y los **permisos** (lo que puede hacer solo, lo que debe preguntar y lo que tiene prohibido). Aquí los revisas y los completas con dos reglas tuyas.

Instrucciones

6.1 — Leer el archivo de contexto. Abre `AGENTS.md` y lee las secciones **Cómo verificar** y **Prohibido**. `CLAUDE.md` solo tiene una línea (`@AGENTS.md`): importa ese archivo para que Claude Code lea las mismas reglas.

6.2 — Escribir dos reglas propias. En la sección **Reglas del estudiante** de `AGENTS.md`, reemplaza `(regla 1)` y `(regla 2)` por dos reglas para tu proyecto, por ejemplo:

```
- Los nombres de clases, funciones y variables van en español, igual que en el resto del código.
- Antes de crear un widget nuevo, revisa si ya existe uno parecido en `lib/widgets/`.
```

6.3 — Abrir el agente. En la terminal D, dentro de la carpeta del proyecto:

```
claude
```

La primera vez en una carpeta pregunta si **confías en ella**: acepta. Sin esa confianza Claude Code **ignora** las reglas de `.claude/settings.json` .

6.4 — Ver los permisos. En el agente escribe `/permissions` : debes ver que puede ejecutar `flutter analyze` y `flutter test` sin preguntar, y que tiene **prohibido** leer `.env` , hacer `git push` y crear etiquetas.

6.5 — Comprobar que leyó tus reglas. Escríbele:

```
¿Qué reglas del proyecto conoces? Cítalas.
```

Debe mencionar cómo verificar (`flutter analyze` y `flutter test`), lo prohibido y **tus dos reglas**.

¿Usas otra herramienta? Los pasos 6 y 7 funcionan igual; cambian los nombres:

	Claude Code	Codex CLI	Antigravity (agy)
Abrirla	`claude`	`codex`	`agy`
Leer reglas	`CLAUDE.md` (importa `AGENTS.md`)	`AGENTS.md`	`AGENTS.md` o `GEMINI.md`
Modo plan	`Shift + Tab` hasta «plan mode on»	`/plan`	`/plan`
Permisos	`/permissions`	`/permissions`	`/permissions`

La tabla completa, con *hooks* y comandos sin interfaz, está en `docs/arnes-paso-a-paso.md`.

Resultado esperado

El agente está abierto en tu proyecto, conoce las reglas de `AGENTS.md` (incluidas las tuyas) y sus permisos son los esperados: analizar y probar sí; leer `.env`, subir código y crear etiquetas no.

Errores comunes

- **El agente no menciona tus reglas**. *Solución:* guarda `AGENTS.md` y abre una conversación nueva; en Claude Code, `/memory` lista los archivos de contexto cargados y `/context` muestra lo que se cargó al empezar.
- **`/permissions` no muestra las reglas del proyecto**. *Solución:* no aceptaste confiar en la carpeta; sal y vuelve a abrirla, y acepta.
- **No tienes cuenta de ningún agente**. *Solución:* trabaja en pareja (como en la Sesión 1) o sigue la demostración del docente; en la guía docente hay una solución ya generada para comparar.
- **Lo marcado con «⚠» en `docs/arnes-paso-a-paso.md`**. *Solución:* son datos que no se pudieron verificar; confírmalos en tu herramienta.

✅ Antes de continuar, confirma que...

- ☐ `AGENTS.md` tiene tus dos reglas.
- ☐ El agente las cita al preguntarle por las reglas del proyecto.
- ☐ `/permissions` (o el equivalente) muestra que `git push` y la lectura de `.env` están

prohibidos.

Paso 7 — Código con un agente: filtro por categoría (9 min)

Por qué se hace

Un pedido bien hecho tiene cuatro partes: **contexto** (qué reglas respetar), **objetivo** (qué comportamiento nuevo), **restricciones** (qué no tocar) y **criterio de aceptación** (cómo sabremos que está bien). Con eso el agente puede comprobar su propio trabajo. Trabajas en **modo plan**: primero explora y propone; recién cuando aprobas, edita. Y al final **verificas tú**, no solo el agente.

Instrucciones

7.1 — Crear una rama para el cambio. En la terminal C:

```
git checkout -b filtro-categoria
```

7.2 — Pasar a modo plan. En el agente (terminal D) presiona `Shift + Tab` hasta que la barra inferior diga **plan mode on**. (Codex y Antigravity: escribe `/plan`.)

7.3 — Hacer el pedido. Pega este texto en el agente y envíalo:

```
Lee AGENTS.md y respeta sus reglas.

Quiero agregar a la pantalla Gastos un filtro por categoría: una fila de chips justo
encima de la lista, con «Todas» y una opción por cada categoría que aparezca en los
gastos. Al tocar una categoría, la lista muestra solo los gastos de esa categoría;
«Todas» los muestra todos.

Criterios de aceptación:
1. `flutter analyze` termina sin errores.
2. `flutter test` pasa completo.
3. Agregas una prueba de widget en `test/filtro_gastos_test.dart` que compruebe que,
al elegir «comida», solo se ven los gastos de comida.

No agregues dependencias ni cambies cómo se conecta la app al backend. Al final dime
qué archivos cambiaste.
```

7.4 — Leer el plan antes de aprobarlo. Comprueba tres cosas: que solo toque el controlador de gastos, la pantalla de gastos y la prueba nueva; que **no agregue paquetes**; y que incluya correr `flutter analyze` y `flutter test`. Si algo no te convence, dile qué cambiar. Si está bien, **apruébalo**.

7.5 — Esperar el cambio. El agente edita los archivos y ejecuta el análisis y las pruebas. Puede tardar uno o dos minutos.

7.6 — Revisar lo que cambió. En la terminal C:

```
git status
git diff
```

Lee el cambio completo: ¿escribió algún secreto?, ¿tocó algo que no pediste?

7.7 — Verificar tú. En la terminal C:

```
flutter analyze
flutter test
```

7.8 — Probarlo en la app. En la terminal B presiona `R`. En la pestaña **Gastos** aparecen los chips: toca una categoría y la lista se filtra; toca «Todas» y vuelve a mostrarse completa.

7.9 — Guardar el cambio. El agente no puede hacer `git push` ni commits por ti; lo haces

tú:

```
git add -A
git commit -m "Sesión 10: filtro de gastos por categoría (generado con un agente)"
```

Resultado esperado

La pantalla Gastos tiene una fila de chips que filtra la lista. `flutter analyze` no muestra errores y `flutter test` pasa **todas** (28 de antes más las que agregó el agente: **29 o más**). El código del agente puede ser distinto al de tus compañeros: lo que importa es que cumpla el criterio de aceptación.

Errores comunes

- **El agente no entra en modo plan y empieza a editar**. *Solución:* detenlo (`Esc`), vuelve a pasar a modo plan con `Shift + Tab` y repite el pedido.
- **El plan o el cambio agrega una dependencia**. *Solución:* recházalo y recuérdale la restricción: «sin dependencias nuevas».
- **`flutter test` falla después del cambio**. *Solución:* pega el mensaje del error al agente y pídele que lo corrija, y vuelve a verificar tú.
- **El agente intenta hacer `git push`**. *Solución:* está prohibido en `.claude/settings.json`; es lo esperado. El push lo haces tú en el Paso 8.
- **Los chips se ven pero «Todas» no restablece la lista**. *Solución:* dile exactamente eso al agente, con el comportamiento esperado, y que agregue una prueba.

✅ Antes de continuar, confirma que...

- ☐ Leíste el plan y el `git diff` antes de aceptar el cambio.
- ☐ `flutter analyze` sin errores y `flutter test` con todas las pruebas pasando.
- ☐ El filtro funciona en la app y el cambio está en un commit de la rama `filtro-categoria`.

Paso 8 — Entrega continua: integrar y publicar `v1.0.0` (7 min)

Por qué se hace

El cambio ya pasó tus pruebas; ahora tiene que pasar la **puerta de calidad** en GitHub, y una

persona (tú) decide integrarlo. Después una **etiqueta de versión** dispara la **entrega continua (CD)**: el workflow «Release APK» vuelve a verificar, compila el APK de release y lo publica en un Release.

Instrucciones

8.1 — Subir la rama. En la terminal C:

```
git push -u origin filtro-categoria
```

8.2 — Abrir el pull request. En tu repositorio en GitHub aparece el botón **Compare & pull request**: tócalo, deja `main` como destino y crea el pull request.

8.3 — Esperar la puerta de calidad. En el pull request, la sección de verificaciones muestra **CI** (verás dos: una por la subida y otra por el pull request). Espera la ✓ verde (unos 2 a 3 minutos). Si sale ✗, abre el detalle y corrige como en el Paso 5.

8.4 — Integrar. Con la CI en verde, toca **Merge pull request** y confirma.

8.5 — Traer `main` actualizado. En la terminal C:

```
git checkout main
git pull
```

8.6 — Crear y subir la etiqueta. Esto dispara el release:

```
git tag v1.0.0
git push origin v1.0.0
```

8.7 — Mirar el release. En GitHub, pestaña **Actions**, abre la ejecución «Release APK». Tarda unos **10 minutos** la primera vez (compilar el APK es lo más lento); mientras tanto, sigue con el Paso 9.

Resultado esperado

El pull request tiene la CI en verde y quedó integrado en `main`. Aparece una ejecución «Release APK» que pasa de amarillo (en curso) a verde (✓) y, al terminar, el repositorio tiene un Release `v1.0.0` con el APK adjunto.

Errores comunes

- **No aparece el botón «Compare & pull request»**. *Solución:* abre la pestaña **Pull**

requests → New pull request, elige `filtro-categoria` y crea el pull request.

- **No quieres esperar la CI**. *Solución:* no la saltes: esperar es el sentido de la puerta. Mientras corre, lee el Paso 9.
- **No aparece ninguna ejecución de «Release APK»**. *Solución:* la etiqueta se subió antes que el workflow, o no estás en `main` actualizado. Sube el commit, y vuelve a crear la etiqueta con otro nombre (`v1.0.1`).
- **El paso «Publicar el Release» falla con `Resource not accessible by integration`**. *Solución:* en GitHub, *Settings → Actions → General → Workflow permissions → Read and write permissions*, y vuelve a ejecutar la etiqueta.
- **Falla «Ejecutar las pruebas» o «Analizar el código»**. *Solución:* abre el paso en Actions y lee el mensaje: es lo mismo que verías en tu computador.

✅ Antes de continuar, confirma que...

- ☐ El pull request se integró con la CI en verde.
- ☐ La etiqueta `v1.0.0` está en GitHub.
- ☐ Ves la ejecución «Release APK» en la pestaña Actions.

Paso 9 — Mientras se publica; luego, descargar e instalar el APK (11 min)

Por qué se hace

El APK es el producto final de la entrega continua. Lo descargas del Release, lo instalas en el emulador como lo haría cualquier persona y compruebas que **el filtro que escribió el agente está dentro de la versión publicada**. Como el release tarda, primero aprovechas la espera.

Instrucciones

9.1 — Aprovechar la espera. Si sobra tiempo, haz el Paso 10 («Acerca de») o el Paso 11 (un *hook*), o lee la plantilla `docs/README-plantilla.md` y la lista `docs/lista-de-verificacion.md`.

9.2 — Esperar el Release. Cuando la ejecución «Release APK» termine en verde, en tu repositorio abre **Releases → v1.0.0**.

9.3 — Descargar el APK. Descarga `app-release.apk`.

9.4 — Desinstalar la versión de depuración. El APK de Actions se firma con otra llave que

la que instala `flutter run`, y Android no deja instalar una encima de la otra. En la terminal C:

```
adb uninstall com.example.exploraec
```

9.5 — Instalar el APK del Release. Con el emulador encendido (ajusta la ruta a donde se descargó):

```
adb install -r ~/Descargas/app-release.apk
```

9.6 — Probarlo. Abre ExploraEC desde el ícono del emulador (esta versión no muestra la franja «DEBUG»), inicia sesión y en **Gastos** usa los chips de categoría.

Resultado esperado

El Release tiene el APK, se instala en el emulador, arranca contra el backend encendido y muestra los chips de categoría.

Errores comunes

- `adb: command not found`. *Solución:* agrega la carpeta `platform-tools` del SDK de Android al PATH, o ejecuta el comando desde esa carpeta.
- `INSTALL_FAILED_UPDATE_INCOMPATIBLE`. *Solución:* quedó instalada la versión de depuración; ejecuta el comando del 9.4 y vuelve a instalar.
- **La app instalada no inicia sesión**. *Solución:* el APK usa la dirección del emulador (`10.0.2.2`); enciende el backend como en el Paso 0 y ejecútalo en el emulador.
- **El APK no muestra los chips**. *Solución:* el Release se generó antes de integrar el cambio; confirma que `v1.0.0` apunta al commit integrado (`git log -1 v1.0.0`).

✅ Antes de continuar, confirma que...

- ☐ La ejecución «Release APK» terminó en verde.
- ☐ Instalaste y abriste el APK del Release, con los chips de categoría.

Paso 10 (opcional) — «Acerca de» y licencias (6 min)

> Este paso **no cuenta dentro de los 60 minutos**. Misma mecánica: descomentar y

> probar.

Por qué se hace

Una app usa código de terceros (paquetes, mapas, datos). Reconocerlos es parte de entregar bien. Flutter ya reúne las licencias de todos los paquetes en una pantalla lista para mostrar.

Instrucciones

10.1 — Opción del menú. En `lib/screens/gastos_screen.dart`, descomentar el bloque `TODO(sesion-10) OPCIONAL Paso 10` que está en `itemBuilder` (1 línea).

10.2 — Acción. En el mismo archivo, descomentar el bloque `TODO(sesion-10) OPCIONAL Paso 10` que está en `onSelected` (1 línea).

10.3 — Aplicar. En la terminal B, presionar `R`.

10.4 — Probar. En Gastos, menú ⋮ → **Acerca de**: aparece la pantalla de licencias con los paquetes que usa la app.

Resultado esperado

La pantalla de licencias lista los paquetes de ExploraEC.

Errores comunes

- **No aparece «Acerca de» en el menú**. *Solución:* falta descomentar el bloque de `itemBuilder`.
- **Aparece pero no pasa nada**. *Solución:* falta el bloque de `onSelected`.

✅ Antes de continuar, confirma que...

- ☐ «Acerca de» abre la pantalla de licencias.

Paso 11 (opcional) — Un *hook* que formatea tras cada edición (6 min)

> Este paso **no cuenta dentro de los 60 minutos** y es específico de Claude Code. La parte equivalente para Codex y Antigravity está en `docs/arnes-paso-a-paso.md`.

Por qué se hace

Una regla en el archivo de contexto **guía** al agente, pero él puede olvidarla. Un *hook* **obliga**: es un comando que se ejecuta siempre en un momento fijo. Aquí, cada vez que el agente edite o cree un archivo, se formatea el código solo.

Instrucciones

11.1 — Abrir la configuración. Abre `.claude/settings.json`.

11.2 — Agregar el hook. Después del bloque `"permissions": { ... }` agrega una coma y este bloque (copia el que está en `docs/arnes-paso-a-paso.md`, sección «Paso 11»):

```
"hooks": {
  "PostToolUse": [
    {
      "matcher": "Edit|Write",
      "hooks": [{ "type": "command", "command": "dart format lib test" }]
    }
  ]
}
```

11.3 — Comprobarlo. En el agente escribe `/hooks`: debe aparecer el hook bajo `PostToolUse`.

11.4 — Probarlo. Pídele al agente: «Agrega un comentario de una línea sobre el filtro en la pantalla de gastos». Después ejecuta `git diff`: el archivo quedó formateado.

Resultado esperado

`/hooks` muestra el hook, y tras cada edición del agente el código queda formateado sin que se lo pidas.

Errores comunes

- **`/hooks` no muestra nada**. *Solución:* el JSON quedó mal (falta una coma o una llave); revisa que el archivo sea JSON válido.
- **El formateo no ocurre**. *Solución:* el agente debe haber editado con sus herramientas de edición; un comando de terminal no activa este hook.

✅ Antes de continuar, confirma que...

- ☐ `/hooks` muestra el hook de formateo.

Llevar el arnés a tu Proyecto Final

Esta parte **no se hace en clase**. Con tu Proyecto Final en otra carpeta (ajusta las rutas):

1. **Copiar el arnés y los workflows** (en PowerShell, `Copy-Item -Recurse`):

```
cp -r exploraec/.github tu-proyecto/
cp -r exploraec/.claude tu-proyecto/
cp exploraec/AGENTS.md exploraec/CLAUDE.md tu-proyecto/
cp -r exploraec/docs tu-proyecto/
```

2. **Adaptar** `AGENTS.md` a tu proyecto (qué es, dónde está cada cosa, tus reglas) y subirlo.
3. **Subir una rama** y comprobar que la CI se pone en verde.
4. **Completar la lista de verificación** (`docs/lista-de-verificacion.md`) y el **README** (`docs/README-plantilla.md`).
5. **Hacer la auditoría de secretos** (Paso 4) en tu repositorio.
6. **Publicar el release** (`git tag v1.0.0` y `git push origin v1.0.0`).

Entregable de la práctica

- Errores no controlados capturados en un solo lugar, sin tokens en los registros.
- 28 pruebas de la base más las del agente (29 o más) pasando.
- Auditoría de secretos limpia: ninguna clave, `.env` ignorado, sin tokens en logs.
- Una CI en verde y un fallo en rojo leído y entendido.
- `AGENTS.md` con tus dos reglas y los permisos del agente revisados.
- Un filtro de gastos por categoría generado con un agente, **leído y verificado por ti**, integrado con un pull request.
- Release `v1.0.0` en GitHub con el APK generado por Actions, instalado y probado en el emulador.
- *(Opcional)* «Acerca de» con licencias (Paso 10) y un *hook* de formateo (Paso 11).

Conexión con el resto del curso

Esta es la última sesión: no hay una siguiente. La conexión es hacia atrás: cada pieza que hoy revisaste se enseñó en una sesión concreta (estado en la 4, errores en la 6, almacenamiento en la 7, sesión y secretos en la 8, agentes, MCP y el asistente en la 9). Lo nuevo de hoy es el **arnés**: la CI, los permisos y las pruebas que permiten usar un agente de código sin perder el control. Tu Proyecto Final es donde se aplican todas juntas.

Anexo — Todos los comandos de la práctica, en orden

Resumen de consulta (Linux/Mac). Las ediciones de archivo en los Pasos 2, 3 y 10 son «descomentar el bloque `TODO(sesion-10) Paso N`»: no son comandos. `<id>` es el que muestra `flutter emulators`.

Paso 0

```
git status
git add -A
git commit -m "Sesión 9"
docker compose up                       # terminal A, en proyecto-curso-spec-kit
curl http://localhost:8000/health       # terminal C
flutter emulators --launch <id>
flutter run                             # terminal B
```

Punto de partida y Paso 1

```
git fetch starter
git checkout starter/sesion-10 -- lib test pubspec.yaml PLACEHOLDERS.md docs .github
.claude AGENTS.md CLAUDE.md analysis_options.yaml
flutter pub get
flutter test                            # 24 pasan, 1 falla
```

Pasos 2, 3 y 4

```
flutter test                          # 25 pasan tras el Paso 2; 28 tras el Paso 3
docker compose up                     # Paso 3, tras apagar el backend con Ctrl + C
grep -rniE "api[_-]?key|sk-[a-z0-9]{10}|SECRET_KEY" lib test .github AGENTS.md
grep -rnE "(debugPrint|print)\(.*(token|Bearer)" lib
git check-ignore -v .env
git log --all --oneline -- .env
```

Paso 5

```
git checkout -b cierre-s10
git add -A
git commit -m "Sesión 10: errores, pruebas, auditoría y CI"
git push -u origin cierre-s10
git checkout -b prueba-roja           # cambiar 'abc' por 'abcdefgh' en test/
sesion_10_test.dart
git commit -am "Prueba rota a propósito"
git push -u origin prueba-roja
git checkout cierre-s10
git branch -D prueba-roja
git push origin --delete prueba-roja
```

Pasos 6 y 7

```
claude                                # terminal D (aceptar la confianza en la
carpeta)
git checkout -b filtro-categoria
git status
git diff
flutter analyze
flutter test
git add -A
git commit -m "Sesión 10: filtro de gastos por categoría (generado con un agente)"
```

Pasos 8 y 9

```
git push -u origin filtro-categoria    # luego: pull request, CI en verde y «Merge
pull request»
git checkout main
git pull
git tag v1.0.0
git push origin v1.0.0
adb uninstall com.example.exploraec
adb install -r ~/Descargas/app-release.apk
```